Платформено-независими програмни езици

Лекция 6

Stream API и функционален стил

Потоци от данни, междинни и крайни операции, колектори, Optional и примитивни потоци

проф. д-р Даниела Гоцева
Учебна 2026/2027 г.



Съдържание на лекцията

1 Функционален стил в Java

2 Потоци – основни понятия

3 Междинни операции

4 Крайни операции

5 Колектори

6 Класът Optional

7 Примитивни и паралелни потоци

8 Пример и добри практики

01

Функционален стил в Java

От „как“ към „какво“

Императивен и декларативен стил

Императивен – „как“

- цикли, броячи, временни променливи
- променяме състояние стъпка по стъпка
- лесно се греши в детайлите
- `for`, `if`, `+=`

Декларативен – „какво“

- описваме **резултата**
- верига от операции
- по-кратко и по-четимо
- `filter`, `map`, `collect`

Една задача – два стила

```java
List<String> names = List.of("мария", "иван", "ана", "георги", "ина");

// Императивно: имената с дължина > 3, с главни букви, сортирани
List<String> result = new ArrayList<>();
for (String n : names) {
    if (n.length() > 3) {
        result.add(n.toUpperCase());
    }
}
Collections.sort(result);

// Със Stream API
List<String> result2 = names.stream()
        .filter(n -> n.length() > 3)
        .map(String::toUpperCase)
        .sorted()
        .toList();
// [ГЕОРГИ, ИВАН, МАРИЯ]
```

Принципи на функционалния стил

- **Чисти функции** – резултатът зависи само от аргументите, без странични ефекти
- **Неизменяемост** – не променяме данни, създаваме нови
- **Функции като стойности** – ламбда изрази и референции към методи
- **Композиция** – сложни операции от прости
- Java не е чисто функционален език, но поддържа този стил от Java 8
- Не е „или-или“ – комбинираме ООП и функционален стил

Python comprehensions и Java потоци

Python

- `[n.upper() for n in names if len(n) > 3]`
- `sum(x * x for x in nums)`
- `{w: len(w) for w in words}`
- `any(x < 0 for x in nums)`

Java

- `names.stream().filter(n -> n.length() > 3).map(String::toUpperCase).toList()`
- `nums.stream().mapToInt(x -> x * x).sum()`
- `words.stream().collect(toMap(w -> w, String::length))`
- `nums.stream().anyMatch(x -> x < 0)`

02

Потоци – основни понятия

Източник, междинни операции, крайна операция

Какво е поток (Stream)?

- **Последователност от елементи**, върху която прилагаме операции
- Потокът **не съхранява** данни – той ги пренася от източник
- Потокът **не променя** източника
- Обработката е **мързелива** (lazy) – нищо не се случва до крайната операция
- Потокът се използва **еднократно**
- Не бъркайте с входно-изходните потоци (`InputStream`) – друго понятие!

Конвейер (pipeline)

- 1. **Източник** – колекция, масив, файл, генератор
- 2. **Междинни операции** – връщат нов поток: `filter`, `map`, `sorted`…
  - може да са нула или много
- 3. **Крайна операция** – произвежда резултат: `toList`, `count`, `forEach`…
  - точно една, затваря потока
- `източник.stream().filter(...).map(...).collect(...)`

Създаване на потоци

- От колекция: `list.stream()`
- От стойности: `Stream.of(...)`
- От масив: `Arrays.stream(arr)`
- Числови диапазони: `IntStream.range`
- От файл: `Files.lines(path)`
- Безкрайни: `Stream.iterate`, `Stream.generate`

```
Stream<String> s1 = List.of("a", "b").stream();
Stream<String> s2 = Stream.of("x", "y", "z");
IntStream s3 = Arrays.stream(new int[]{1, 2});
IntStream s4 = IntStream.range(0, 10); // 0..9
IntStream s5 = IntStream.rangeClosed(1, 10);

Stream<String> lines =
        Files.lines(Path.of("data.txt"));

Stream<Integer> pow2 =
        Stream.iterate(1, x -> x * 2);
Stream<Double> rnd =
        Stream.generate(Math::random);
```

Stream.iterate с условие

- От Java 9: `iterate(начало, условие, следващ)`
- Като `for` цикъл, но поток
- Без нужда от `limit` или `takeWhile`

```java
// като for (int i = 1; i <= 1000; i *= 2)
Stream.iterate(1, i -> i <= 1000, i -> i * 2)
      .forEach(System.out::println);

// Числа на Фибоначи до 100
Stream.iterate(new long[]{0, 1},
               f -> f[0] <= 100,
               f -> new long[]{f[1], f[0] + f[1]})
      .map(f -> f[0])
      .toList();
// [0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
```

Мързеливо изпълнение

- Междинните операции само **описват** обработката
- Изпълнението започва с крайната операция
- Елементите минават през конвейера **един по един**
- Позволява работа с безкрайни потоци

```
Stream<String> s = Stream.of("a", "bb", "ccc")
        .filter(x -> {
            System.out.println("filter " + x);
            return x.length() > 1;
        });
// нищо не е отпечатано досега!

s.forEach(x -> System.out.println("got " + x));
// filter a
// filter bb
// got bb
// filter ccc
// got ccc
```

Потокът е за еднократна употреба

- След крайната операция потокът е **затворен**
- Повторно използване → `IllegalStateException`
- Ако ни трябва отново – създаваме нов поток от източника

```
Stream<String> s =
        Stream.of("a", "b", "c");

long n = s.count();      // 3
// s.forEach(System.out::println);
//   -> IllegalStateException:
//      stream has already been operated
//      upon or closed

List<String> src = List.of("a", "b", "c");
long n1 = src.stream().count();
src.stream().forEach(System.out::println);
```

Ред на операциите и ефективност

- Филтрирайте **рано** – по-малко елементи за следващите стъпки
- `sorted` изисква всички елементи – слагайте го след `filter`
- `limit` след `sorted` – „топ N“
- Short-circuit операциите (`findFirst`, `anyMatch`, `limit`) спират рано

```
// По-бавно: сортира всички, после филтрира
products.stream()
        .sorted(byPrice)
        .filter(p -> p.price() > 100)
        .toList();

// По-бързо: филтрира, после сортира малко
products.stream()
        .filter(p -> p.price() > 100)
        .sorted(byPrice)
        .toList();
```

03

Междинни операции

Филтриране, преобразуване, сортиране, ограничаване

Основни междинни операции

Операция	Действие	Пример
`filter(p)`	оставя елементите по условие	`filter(x -> x > 0)`
`map(f)`	преобразува всеки елемент	`map(String::length)`
`flatMap(f)`	елемент → поток; потоците се сливат	`flatMap(List::stream)`
`distinct()`	премахва повторенията	`distinct()`
`sorted()`, `sorted(cmp)`	сортира	`sorted(Comparator.reverseOrder())`
`limit(n)`, `skip(n)`	първите n / пропуска n	`limit(10)`
`takeWhile(p)`, `dropWhile(p)`	взема / пропуска, докато е вярно	`takeWhile(x -> x < 9)`

filter и map

- `filter` – намалява броя на елементите
- `map` – сменя стойността (и типа) на елементите
- Може да се редуват многократно

```
record Product(String name, double price,
               String category) { }

List<Product> products = List.of(
    new Product("Лаптоп", 1800, "техника"),
    new Product("Мишка", 35, "техника"),
    new Product("Книга", 25, "книги"),
    new Product("Монитор", 420, "техника"));

List<String> expensive = products.stream()
        .filter(p -> p.price() > 100)
        .map(Product::name)
        .toList();
// [Лаптоп, Монитор]
```

flatMap – „изравняване“

- Всеки елемент се превръща в **поток**
- Получените потоци се сливат в един
- Типично: списък от списъци → един списък; текст → думи

```
List<List<Integer>> nested = List.of(
        List.of(1, 2),
        List.of(3),
        List.of(4, 5, 6));

List<Integer> flat = nested.stream()
        .flatMap(List::stream)
        .toList();
// [1, 2, 3, 4, 5, 6]

List<String> lines = List.of(
        "Java е език", "Java е платформа");
long words = lines.stream()
        .flatMap(l -> Arrays.stream(l.split(" ")))
        .distinct()
        .count();       // 4
```

sorted, distinct, limit и skip

- `sorted` – естествена наредба или `Comparator`
- `distinct` – по `equals`
- `limit` и `skip` – за „страниране“ и „топ N“

```
List<Integer> nums =
        List.of(5, 3, 9, 3, 1, 9, 7);

nums.stream().distinct().sorted().toList();
// [1, 3, 5, 7, 9]

nums.stream()
    .sorted(Comparator.reverseOrder())
    .limit(3)
    .toList();
// [9, 9, 7] — топ 3

// Страница 2 по 3 елемента
nums.stream().skip(3).limit(3).toList();
// [3, 1, 9]
```

takeWhile и dropWhile

- Работят с **подредени** потоци
- `takeWhile` – взема елементите, **докато** условието е вярно, после спира
- `dropWhile` – пропуска, докато условието е вярно
- За разлика от `filter` не проверяват останалите елементи

```
List<Integer> temps =
        List.of(12, 15, 18, 25, 14, 30);

temps.stream()
     .takeWhile(t -> t < 20)
     .toList();     // [12, 15, 18]

temps.stream()
     .dropWhile(t -> t < 20)
     .toList();     // [25, 14, 30]

Stream.iterate(1, x -> x * 3)
      .takeWhile(x -> x < 1000)
      .toList();    // [1, 3, 9, 27, 81, 243, 729]
```

Отстраняване на грешки с peek

- `peek` вижда елементите, **без да ги променя**
- Удобно, за да проследим какво минава през конвейера
- В Eclipse: точка на прекъсване в ламбдата
- Не използвайте `peek` за бизнес логика

```
List<String> r = Stream.of("ана", "иван", "ив")
        .peek(s -> System.out.println("вход: " + s))
        .filter(s -> s.length() > 2)
        .peek(s -> System.out.println("след filter: " + s))
        .map(String::toUpperCase)
        .toList();
```

Потоци от низове и Map

- `chars()` – `IntStream` от символите на низ
- `entrySet().stream()` – поток от двойките на `Map`
- `Map.Entry.comparingByValue()` – сортиране по стойност

```
long vowels = "Програмиране".toLowerCase()
        .chars()
        .filter(c -> "аеиоуъюя".indexOf(c) >= 0)
        .count();                                  // 5

Map<String, Integer> freq =
        Map.of("java", 5, "python", 8, "c", 2);

freq.entrySet().stream()
    .sorted(Map.Entry.<String, Integer>
            comparingByValue().reversed())
    .limit(2)
    .forEach(e -> System.out.println(
            e.getKey() + "=" + e.getValue()));
// python=8
// java=5
```

Сортиране по няколко критерия

- `sorted(Comparator)` – с всички възможности на `Comparator`
- `thenComparing` – вторичен ключ
- `reversed()` – обратен ред

```
record Student(String name, int course,
               double grade) { }

List<Student> list = List.of(
    new Student("Вера", 2, 5.5),
    new Student("Ана", 1, 6.0),
    new Student("Боян", 2, 6.0));

list.stream()
    .sorted(Comparator
        .comparingInt(Student::course)
        .thenComparing(Student::grade,
                Comparator.reverseOrder()))
    .map(Student::name)
    .toList();
// [Ана, Боян, Вера]
```

04

Крайни операции

Произвеждане на резултат

Основни крайни операции

Операция	Резултат	Пример
`toList()`	неизменяем `List` (Java 16)	`.toList()`
`collect(collector)`	колекция, низ, `Map`…	`.collect(Collectors.toSet())`
`forEach(action)`	действие за всеки елемент	`.forEach(System.out::println)`
`count()`	брой елементи (`long`)	`.count()`
`min(cmp)`, `max(cmp)`	`Optional<T>`	`.max(Comparator.naturalOrder())`
`reduce(...)`	обединяване в една стойност	`.reduce(0, Integer::sum)`
`anyMatch`, `allMatch`, `noneMatch`	`boolean`	`.anyMatch(x -> x < 0)`
`findFirst()`, `findAny()`	`Optional<T>`	`.findFirst()`

count, min, max и проверки

- `min`/`max` връщат `Optional` – потокът може да е празен
- `anyMatch` и подобните спират веднага щом знаят отговора (short-circuit)

```
long cheap = products.stream()
        .filter(p -> p.price() < 50)
        .count();                          // 2

Optional<Product> top = products.stream()
        .max(Comparator.comparingDouble(
                Product::price));
top.ifPresent(p ->
        System.out.println(p.name())); // Лаптоп

boolean anyBooks = products.stream()
        .anyMatch(p -> p.category()
                        .equals("книги")); // true

boolean allPositive = products.stream()
        .allMatch(p -> p.price() > 0);     // true
```

reduce – свиване до една стойност

- Комбинира елементите един по един
- `reduce(начална, операция)` – с неутрален елемент
- `reduce(операция)` – връща `Optional`
- Операцията трябва да е **асоциативна**

```java
List<Integer> nums = List.of(1, 2, 3, 4, 5);

int sum = nums.stream()
        .reduce(0, Integer::sum);          // 15

int product = nums.stream()
        .reduce(1, (a, b) -> a * b);       // 120

Optional<Integer> max = nums.stream()
        .reduce(Integer::max);             // 5

String joined = Stream.of("a", "b", "c")
        .reduce("", (x, y) -> x + y);      // "abc"
```

findFirst и forEach

- `findFirst` – първият елемент, отговарящ на условието
- Връща `Optional` – може да няма такъв
- `forEach` – за странични ефекти: печат, запис
- Не използвайте `forEach` за пълнене на външни колекции – има `collect`

```
Optional<Product> firstBook =
    products.stream()
        .filter(p -> p.category()
                      .equals("книги"))
        .findFirst();

String name = firstBook
        .map(Product::name)
        .orElse("няма");           // Книга

products.stream()
        .sorted(Comparator.comparing(
                Product::price))
        .forEach(p -> System.out.printf(
                "%-10s %8.2f%n",
                p.name(), p.price()));
```

05

Колектори

Събиране на резултата в колекции, низове и групи

Класът Collectors

Колектор	Резултат
`toList()`, `toSet()`	списък, множество
`toMap(key, value)`	`Map`
`joining(", ")`	слепен низ с разделител
`groupingBy(f)`	`Map<K, List<T>>` – групи по ключ
`partitioningBy(p)`	`Map<Boolean, List<T>>` – две групи
`counting()`	брой елементи (в група)
`averagingDouble(f)`, `summingInt(f)`	средно, сума
`mapping(f, collector)`	преобразува преди събиране

Събиране в конкретна колекция

- `toList()` – неизменяем списък
- `toCollection(Supplier)` – избираме реализацията
- `toUnmodifiableSet()` – неизменяемо множество

```
TreeSet<String> sorted = products.stream()
        .map(Product::category)
        .collect(Collectors.toCollection(
                TreeSet::new));
// [книги, техника]

ArrayList<String> editable = products.stream()
        .map(Product::name)
        .collect(Collectors.toCollection(
                ArrayList::new));
editable.add("Нов");     // позволено

Set<String> fixed = products.stream()
        .map(Product::name)
        .collect(Collectors.toUnmodifiableSet());
```

joining и toMap

- `joining` – елегантно слепване на низове
- `toMap` – ключ и стойност от всеки елемент
- При повтарящ се ключ `toMap` хвърля изключение – трети аргумент решава конфликта

```
String csv = products.stream()
        .map(Product::name)
        .collect(Collectors.joining(", ",
                "[", "]"));
// [Лаптоп, Мишка, Книга, Монитор]

Map<String, Double> priceOf =
    products.stream()
        .collect(Collectors.toMap(
                Product::name,
                Product::price));

Map<String, Double> maxByCat =
    products.stream()
        .collect(Collectors.toMap(
                Product::category,
                Product::price,
                Math::max));
```

groupingBy – групиране

- Разделя елементите на групи по ключ
- Резултат: `Map<ключ, List<елемент>>`
- Втори аргумент – какво да се събере за всяка група

```
Map<String, List<Product>> byCat =
    products.stream()
        .collect(Collectors.groupingBy(
                Product::category));
// {книги=[Книга], техника=[Лаптоп, Мишка, Монитор]}

Map<String, Long> countByCat =
    products.stream()
        .collect(Collectors.groupingBy(
                Product::category,
                Collectors.counting()));
// {книги=1, техника=3}
```

groupingBy с вложени колектори

```
// Средна цена по категория, с подредени ключове
Map<String, Double> avg = products.stream()
        .collect(Collectors.groupingBy(
                Product::category,
                TreeMap::new,
                Collectors.averagingDouble(Product::price)));
// {книги=25.0, техника=751.67}

// Само имената по категория
Map<String, List<String>> names = products.stream()
        .collect(Collectors.groupingBy(
                Product::category,
                Collectors.mapping(Product::name, Collectors.toList())));
// {книги=[Книга], техника=[Лаптоп, Мишка, Монитор]}
```

Пример: честота на думите

- Цялата задача от миналата лекция – в един поток
- `groupingBy` + `counting`
- Сортиране по брой, топ 3

```
String text = "java е език java е платформа "
            + "python е език";

Arrays.stream(text.split("\\s+"))
      .collect(Collectors.groupingBy(
              w -> w, Collectors.counting()))
      .entrySet().stream()
      .sorted(Map.Entry.<String, Long>
              comparingByValue().reversed())
      .limit(3)
      .forEach(e -> System.out.println(
              e.getKey() + ": " + e.getValue()));
// е: 3
// java: 2
// език: 2
```

partitioningBy – разделяне на две

- Специален случай на групиране – по условие
- Винаги има ключове `true` и `false`
- Може да се комбинира с друг колектор

```
record Student(String name, double grade) { }

List<Student> st = List.of(
        new Student("Ана", 5.5),
        new Student("Боян", 2.0),
        new Student("Вера", 4.0),
        new Student("Гошо", 2.5));

Map<Boolean, List<String>> passed =
    st.stream().collect(
        Collectors.partitioningBy(
            s -> s.grade() >= 3,
            Collectors.mapping(Student::name,
                    Collectors.toList())));
// {false=[Боян, Гошо], true=[Ана, Вера]}
```

Статистики

- `summarizingDouble` – брой, сума, мин, макс и средно наведнъж
- `teeing` – два колектора едновременно, резултатите се комбинират

```
DoubleSummaryStatistics stats =
    st.stream().collect(
        Collectors.summarizingDouble(
                Student::grade));

stats.getCount();     // 4
stats.getAverage();   // 3.5
stats.getMax();       // 5.5

double range = st.stream().collect(
    Collectors.teeing(
        Collectors.summarizingDouble(Student::grade),
        Collectors.counting(),
        (s, n) -> s.getMax() - s.getMin()));
// 3.5
```

06

Класът Optional

Явно моделиране на „може да няма стойност“



Optional казва явно: „резултатът може да липсва“

Вместо да връщаме `null` и да се надяваме, че извикващият ще провери, връщаме `Optional<T>`. Компилаторът и четящият кода виждат, че стойност може да няма.

Създаване и проверка на Optional

- `Optional.of(x)` – със стойност (x не е `null`)
- `Optional.ofNullable(x)` – x може да е `null`
- `Optional.empty()` – празен
- `isPresent()`, `isEmpty()`, `ifPresent(...)`

```
static Optional<Student> find(
        List<Student> list, String name) {
    return list.stream()
            .filter(s -> s.name().equals(name))
            .findFirst();
}

Optional<Student> r = find(st, "Ана");
r.ifPresent(s ->
        System.out.println(s.grade()));

if (find(st, "Иван").isEmpty()) {
    System.out.println("Няма такъв студент");
}
```

Извличане на стойността

- `orElse(x)` – стойност по подразбиране
- `orElseGet(() -> ...)` – изчислява се само при нужда
- `orElseThrow()` – изключение, ако е празен
- `map`, `filter` – работа без разопаковане
- **Избягвайте** `get()` без проверка

```
String grade = find(st, "Ана")
        .map(s -> "Оценка: " + s.grade())
        .orElse("няма данни");

double g = find(st, "Вера")
        .map(Student::grade)
        .orElseThrow();

Student s = find(st, "Иван")
        .orElseGet(() ->
            new Student("Иван", 2));

find(st, "Ана")
        .filter(x -> x.grade() >= 5.5)
        .ifPresentOrElse(
            x -> System.out.println("Отличен"),
            () -> System.out.println("—"));
```

Верижна работа с Optional

- `map` – преобразува стойността, ако я има
- `flatMap` – когато функцията сама връща `Optional`
- `or` – алтернативен `Optional`
- Без нито една проверка за `null`

```
record Address(String city) { }
record Person(String name,
              Optional<Address> address) { }

Optional<Person> p = findPerson("Ана");

String city = p
        .flatMap(Person::address)
        .map(Address::city)
        .orElse("неизвестен град");

Optional<Person> any = findPerson("Ана")
        .or(() -> findPerson("Боян"));
```

Добри практики с Optional

- Използвайте го като **тип на резултат** от метод
- **Не** го използвайте за полета, параметри или в колекции
- Никога не връщайте `null` вместо `Optional` – връщайте `Optional.empty()`
- Предпочитайте `orElse`, `map`, `ifPresent` пред `isPresent()` + `get()`
- За примитиви: `OptionalInt`, `OptionalDouble`
- В Python аналогът е `Optional[T]` от модула `typing` – но там е само подсказка

07

Примитивни и паралелни потоци

IntStream, DoubleStream, parallel() и Stream Gatherers

Примитивни потоци

- `IntStream`, `LongStream`, `DoubleStream` – без опаковане в обекти
- Методи: `sum`, `average`, `min`, `max`, `summaryStatistics`
- `mapToInt`, `mapToDouble` – от обекти към примитиви
- `boxed()`, `mapToObj` – обратно към обекти

```
int sum = IntStream.rangeClosed(1, 100).sum();
// 5050

double avg = st.stream()
        .mapToDouble(Student::grade)
        .average()
        .orElse(0);     // 3.5

List<Integer> squares = IntStream.range(1, 6)
        .map(x -> x * x)
        .boxed()
        .toList();      // [1, 4, 9, 16, 25]

String stars = IntStream.range(0, 5)
        .mapToObj(i -> "*")
        .collect(Collectors.joining());
```

Масиви и потоци

- `Arrays.stream(int[])` – `IntStream` направо от масив
- `toArray()` – обратно към масив
- Удобни статистики без цикли

```
int[] scores = {72, 95, 60, 88, 41};

int max = Arrays.stream(scores).max().getAsInt();
double avg = Arrays.stream(scores)
                   .average().orElse(0);
long passed = Arrays.stream(scores)
                    .filter(s -> s >= 50)
                    .count();

int[] curved = Arrays.stream(scores)
        .map(s -> Math.min(100, s + 5))
        .toArray();
// [77, 100, 65, 93, 46]
```

Пример: прости числа

- Комбинация от `IntStream`, `filter` и `noneMatch`
- Кодът се чете почти като математическа дефиниция

```
static boolean isPrime(int n) {
    return n > 1 && IntStream
            .rangeClosed(2, (int) Math.sqrt(n))
            .noneMatch(d -> n % d == 0);
}

List<Integer> primes = IntStream
        .rangeClosed(1, 50)
        .filter(Main::isPrime)
        .boxed()
        .toList();
// [2, 3, 5, 7, 11, 13, ..., 47]

long count = IntStream.range(0, 1_000_000)
        .filter(Main::isPrime)
        .count();     // 78498
```

Паралелни потоци

- `parallelStream()` или `.parallel()` – обработка на няколко ядра
- Използва общ пул от нишки (ForkJoinPool)
- Има смисъл при **много** данни и тежки изчисления
- Опасно при споделено изменяемо състояние!
- Измервайте – паралелното не винаги е по-бързо

```
long n = IntStream.range(0, 10_000_000)
        .parallel()
        .filter(Main::isPrime)
        .count();

// ГРЕШНО – състезание между нишките
List<Integer> bad = new ArrayList<>();
IntStream.range(0, 1000).parallel()
        .forEach(bad::add);   // загубени елементи!

// ПРАВИЛНО
List<Integer> ok = IntStream.range(0, 1000)
        .parallel().boxed().toList();
```

Stream Gatherers (Java 24)

- `gather(...)` – собствени **междинни** операции
- Готови в `Gatherers`:
  - `windowFixed(n)` – групи по n
  - `windowSliding(n)` – плъзгащ се прозорец
  - `fold`, `scan` – натрупване

```
Stream.of(1, 2, 3, 4, 5, 6, 7)
      .gather(Gatherers.windowFixed(3))
      .toList();
// [[1, 2, 3], [4, 5, 6], [7]]

Stream.of(1, 2, 3, 4)
      .gather(Gatherers.windowSliding(2))
      .toList();
// [[1, 2], [2, 3], [3, 4]]

Stream.of(1, 2, 3, 4)
      .gather(Gatherers.scan(() -> 0,
                Integer::sum))
      .toList();
// [1, 3, 6, 10] – натрупани суми
```

08

Пример и добри практики

Анализ на данни с потоци

Пример: анализ на оценки (данни)

```
record Result(String student, String subject, int grade) { }

List<Result> results = List.of(
        new Result("Ана",  "Java",   6), new Result("Ана",  "Бази данни", 5),
        new Result("Боян", "Java",   4), new Result("Боян", "Бази данни", 3),
        new Result("Вера", "Java",   5), new Result("Вера", "Бази данни", 6),
        new Result("Гошо", "Java",   2), new Result("Гошо", "Бази данни", 4));
```

Типичен случай: данни от файл или база данни, които трябва да анализираме

Пример: анализ на оценки (заявки)

```
// Среден успех по предмет
Map<String, Double> bySubject = results.stream()
        .collect(Collectors.groupingBy(Result::subject,
                TreeMap::new, Collectors.averagingInt(Result::grade)));
// {Java=4.25, Бази данни=4.5}

// Студентите с поне една шестица
List<String> withSix = results.stream()
        .filter(r -> r.grade() == 6)
        .map(Result::student).distinct().sorted().toList();
// [Ана, Вера]

// Класиране по среден успех
results.stream()
        .collect(Collectors.groupingBy(Result::student,
                Collectors.averagingInt(Result::grade)))
        .entrySet().stream()
        .sorted(Map.Entry.<String, Double>comparingByValue().reversed())
        .forEach(e -> System.out.printf("%-5s %.2f%n", e.getKey(), e.getValue()));
```

Пример: анализ на лог файл

- `Files.lines` – мързеливо четене ред по ред
- Броим грешките по час
- Работи и за файлове от гигабайти

```
// 2026-11-02 14:05:12 ERROR Връзката прекъсна
try (Stream<String> lines =
        Files.lines(Path.of("app.log"))) {

    Map<String, Long> errorsByHour = lines
        .filter(l -> l.contains(" ERROR "))
        .map(l -> l.substring(11, 13))
        .collect(Collectors.groupingBy(
                h -> h, TreeMap::new,
                Collectors.counting()));

    System.out.println(errorsByHour);
    // {09=2, 14=7, 15=1}
}
```

Добри практики със Stream API

- Една операция на ред – по-лесно четене и отстраняване на грешки
- Използвайте референции към методи, където е възможно
- Без странични ефекти в `filter` и `map`
- Не превръщайте всеки цикъл в поток – простият `for` понякога е по-ясен
- `toList()` вместо `collect(Collectors.toList())` (Java 16+)
- Потоци върху файлове – в `try-with-resources` (следващата лекция)
- `parallel()` – само след измерване

Цикъл или поток?

Предпочитайте цикъл, когато...

- има сложна логика с `break` и `continue`
- трябва да хвърляте проверими изключения
- променяте няколко локални променливи
- редът на страничните ефекти е важен

Предпочитайте поток, когато...

- филтрирате, преобразувате, групирате
- резултатът е колекция или една стойност
- искате декларативен, четим код
- данните са много и може да помогне `parallel()`

Типични грешки със Stream API

- Повторно използване на поток → `IllegalStateException`
- Забравена крайна операция – нищо не се изпълнява
- Промяна на източника по време на обработка
- Странични ефекти в `map` и `filter`
- `get()` на празен `Optional` → `NoSuchElementException`
- `toMap` с повтарящи се ключове без функция за сливане
- `parallel()` с обща изменяема колекция

Обобщение

- Поток = източник → междинни операции → крайна операция
- Мързеливо изпълнение; потокът се използва еднократно
- Междинни: `filter`, `map`, `flatMap`, `sorted`, `distinct`, `limit`, `takeWhile`
- Крайни: `toList`, `collect`, `count`, `reduce`, `anyMatch`, `findFirst`
- `Collectors`: `joining`, `toMap`, `groupingBy`, `partitioningBy`
- `Optional` – явно „може да няма стойност“
- `IntStream` и др. – без опаковане; `parallel()` – внимателно
- Gatherers (Java 24) – собствени междинни операции

Въпроси за самопроверка

- Какво означава, че потоците са „мързеливи“?
- Каква е разликата между `map` и `flatMap`?
- Защо `max()` връща `Optional`?
- Как да преброим елементите във всяка група с `groupingBy`?
- Каква е разликата между `filter` и `takeWhile`?
- Кога паралелният поток може да даде грешен резултат?
- Напишете поток, който връща сумата на квадратите на четните числа от 1 до 100.

Въпроси?

Следваща лекция: Изключения и вход/изход

